

INF5130 - Algorithmique

Solution série d'exercices - 1

1 Exercice 1

- a) La première variante suppose que la plus petite puissance de 2 se trouve à la première case des tableaux A, B et C .

```
1 def additionner(A, B):
2     retenue = 0
3     C = [0] * (len(A) + 1)
4     i = 0
5     while i < len(A):
6         C[i] = (A[i] + B[i] + retenue) % 2
7         retenue = (A[i] + B[i] + retenue) // 2
8         i = i + 1
9     C[len(A)] = retenue
10    return C
```

La deuxième variante suppose que la plus petite puissance de 2 se trouve à la dernière position des tableaux A, B et C .

```
1 def additionner(A, B):
2     retenue = 0
3     C = [0] * (len(A) + 1)
4     i = len(A) - 1
5     while i >= 0:
6         C[i + 1] = (A[i] + B[i] + retenue) % 2
7         retenue = (A[i] + B[i] + retenue) // 2
8         i = i - 1
9     C[0] = retenue
10    return C
```

- b) Dans les deux cas, une opération barométrique identifiable est la comparaison effectuée à la ligne 5, qui s'effectue $n + 1$ fois où n est la taille des tableaux A et B . Puisque cette ligne s'exécute en temps constant à chaque fois, la complexité temporelle de la fonction ADDITIONNER est $\mathcal{O}(n)$.

2 Exercice 2

- a) Le tableau C doit maintenant avoir $2n$ éléments.

- b) La première variante suppose que la plus petite puissance de 2 se trouve à la première case des tableaux A, B et C .

```

1  def multiplier(A,B):
2      C = [0] * (len(A))
3      decalage = 0
4      while decalage < len(A):
5          temp = [0] * (len(A) + decalage)
6          j = 0
7          while j < len(A):
8              temp[j + decalage] = A[j] * B[decalage]
9              j = j + 1
10         C = add_bin(temp, C)
11         decalage = decalage + 1
12     return C

```

La deuxième variante suppose que la plus petite puissance de 2 se trouve à la dernière case des tableaux A, B et C .

```

1  def multiplier(A,B):
2      C = [0] * (len(A))
3      decalage = 0
4      while decalage < len(A):
5          temp = [0] * (len(A) + decalage)
6          j = len(A) - 1
7          while j >= 0:
8              temp[len(temp) - decalage - (len(A) - j - 1) - 1] = A[j] * B[decalage - 1]
9              j = j - 1
10         C = add_bin(temp, C)
11         decalage = decalage + 1
12     return C

```

- c) Une opération barométrique possiblement identifiable pour évaluer la complexité de cet algorithme est l'instruction qui concerne l'addition de deux nombres binaires (ligne 10). Étant donné que la taille du tableau C doit avoir $2n$ éléments à la fin de l'algorithme et que ce dernier commence avec n éléments, le coût de cette instruction est de $\sum_{i=n}^{2n-1} (i+1)$ si on inclut le

coût de l'assignation.

$$\begin{aligned}
 \tau(\text{MULTIPLIER}) &= \sum_{i=n}^{2n-1} (i+1) \\
 &= \sum_{i=0}^{\psi} i + \sum_{i=0}^{\psi} 1 - \left(\sum_{i=0}^{n-1} i + \sum_{i=0}^{n-1} 1 \right) \quad \text{posons } \psi = 2n-1 \\
 &= \frac{\psi(\psi+1)}{2} + \psi + \left(\frac{n(n-1)}{2} + n \right) \\
 &= \frac{4n^2 - 2n}{2} + 2n - 1 + 1 - \frac{n^2 + n}{2} \\
 &= \frac{4n^2 - 2n + 4n - n^2 - n}{2} \\
 &= \frac{3}{2}n^2 + \frac{1}{2}n \in \mathcal{O}(n^2)
 \end{aligned}$$

3 Exercice 3

```

1 def contient_cycle(L):
2     lent = obtenir_tete(L)
3     rapide = obtenir_tete(L)
4     if rapide is None:
5         return False
6     cyclique = False
7     while not (rapide is None) and not (obtenir_suivant(rapide) is None) and not cyclique:
8         lent = obtenir_suivant(lent)
9         rapide = obtenir_suivant(rapide)
10        if rapide == lent:
11            cyclique = True
12    return cyclique

```

4 Exercice 4

- a) En premier lieu, voici le nombre de fois où chaque opération est exécutée, ainsi que son coût :

Numéro de la ligne	Nombre de fois exécutée	Coût
1	1	$c_1 = k + 2$
2	1	$c_2 = n + 2$
3	1	c_3
4	$n + 1$	c_4
5	n	c_5
6	n	c_6
7	1	c_7
8	$k + 1$	c_8
9	k	c_9
10	k	c_{10}
11	1	c_{11}
12	$n + 1$	c_{12}
13	n	c_{13}
14	n	c_{14}
15	n	c_{15}
16	1	c_{16}

Le coût total de cette procédure est le suivant

$$\begin{aligned}
\tau(\text{TRICOMPTAGE}) &= c_1 + c_2 + c_3 + \sum_{i=0}^n c_4 + \sum_{i=0}^{n-1} (c_5 + c_6) + c_7 \\
&\quad + \sum_{i=0}^k c_8 + \sum_{i=0}^{k-1} (c_9 + c_{10}) + c_{11} + \sum_{i=0}^n c_{12} \\
&\quad + \sum_{i=0}^{n-1} (c_{13} + c_{14} + c_{15}) + c_{16} \\
&= n(c_4 + c_5 + c_6 + c_{12} + c_{13} + c_{14} + c_{15}) \\
&\quad + k(c_8 + c_9 + c_{10}) + c_1 + c_2 + c_3 + c_4 \\
&\quad + c_7 + c_8 + c_{12} + c_{16} \\
&= n(c_4 + c_5 + c_6 + c_{12} + c_{13} + c_{14} + c_{15} + 1) \\
&\quad + k(c_8 + c_9 + c_{10} + 1) + c_3 + c_4 + c_7 + c_8 \\
&\quad + c_{12} + c_{16} + 4
\end{aligned}$$

Donc, la complexité temporelle est $\mathcal{O}(n + k)$.

Note : le dernier développement est si on développe les coûts c_1 et c_2 qui sont les seuls dont leur coût n'est pas constant à chaque fois qu'on effectue une instance de ces dernières.

b) La complexité spatiale de cet algorithme est $\mathcal{O}(n + k)$.

5 Exercice 5

- a) L'algorithme transforme la matrice passée en paramètre en matrice triangulaire inférieure : pour chaque case de la partie triangulaire supérieure, il ajoute à la case correspondante de la partie triangulaire inférieure (de façon à peu près égale selon la ligne correspondante) la valeur déjà présente dans la case, plus la somme des éléments faisant partie de la partie triangulaire inférieure.
- b) Une opération barométrique possible est l'instruction se trouvant à la ligne 7. En effet, la comparaison est effectuée $\sum_{i=0}^{n-1} \sum_{j=i+1}^n 1$ fois. La raison de ce choix est que, parmi toutes les instructions présentes dans la boucle principale entre les lignes 5 et 19, les instructions qui sont exécutées le plus souvent sont les instructions de comparaison des boucles aux lignes 7 et 14.
- c)

$$\begin{aligned}
 \tau(\text{WERK}) &= \sum_{i=0}^{n-1} \sum_{j=i+1}^n 1 \\
 &= \sum_{i=0}^{n-1} \left(\sum_{j=0}^n 1 - \sum_{j=0}^i 1 \right) \\
 &= \sum_{i=0}^{n-1} ((n+1) - (i+1)) \\
 &= \sum_{i=0}^{n-1} n - \sum_{i=0}^{n-1} i \\
 &= n \times n - \frac{n(n-1)}{2} \\
 &= \frac{n^2 - n}{2} \in \mathcal{O}(n^2)
 \end{aligned}$$

6 Exercice 6

6.1 Approche naïve

a)

```
1 def somme_diagonale(M):
2     somme = 0
3     i = 0
4     while i < len(M):
5         j = 0
6         while j < len(M):
7             if i == j or j = (len(M) - i - 1):
8                 somme = somme + M[i][j]
9             j = j + 1
10        i = i + 1
11    return somme
```

b) Une opération barométrique possible dans cet algorithme est l'instruction se situant à la ligne 6. En effet, l'instruction de comparaison effectuée à l'intérieur de cette boucle imbriquée s'exécute exactement $\sum_{i=0}^{n-1} \sum_{j=0}^n 1$ fois. Comparée à certaines autres instructions, cette dernière est la seule qui s'exécute exactement $\sum_{i=0}^{n-1} \sum_{j=0}^n 1$ fois.

c)

$$\begin{aligned}\tau(\text{SOMMEDIAGONALE}) &= \sum_{i=0}^{n-1} \sum_{j=0}^n 1 \\ &= \sum_{i=0}^{n-1} (n+1) \\ &= (n+1) \sum_{i=0}^{n-1} 1 \\ &= (n+1)n \\ &= n^2 + n \in \mathcal{O}(n^2)\end{aligned}$$

6.2 Approche plus efficace

a)

```
1 def somme_diagonale(M):
2     somme = 0
3     milieu = 0
4     if n % 2 == 1:
5         milieu = n // 2
6     i = 0
7     while i < len(M):
8         somme = somme + M[i][i]
9         i = i + 1
10    j = i - 1
11    i = 0
12    while i < len(M):
13        somme = somme + M[i][j - i]
14        i = i + 1
15    if milieu > 0:
16        somme = somme - M[milieu][milieu]
17    return somme
```

b) Un choix possible pour l'opération barométrique dans cet algorithme est l'instruction de comparaison effectuée dans la boucle de la ligne 7. En effet, excepté l'instruction effectuée à la ligne 12, elle est la seule instruction qui est exécutée exactement n fois, où n représente les dimensions de la matrice M . Le reste des instructions sont exécutées en temps constant, ou bien $n - 1$ fois pour les instructions aux lignes 8 et 13, qui auraient pu être aussi des choix possibles.

c)

$$\begin{aligned}\tau(\text{SOMMEDIAGONALE}) &= \sum_{i=0}^n 1 \\ &= n + 1 \in \mathcal{O}(n)\end{aligned}$$